Linux网络--Http表单与动态路由

- 1. 从静态文件到动态服务
  - 1.1 浏览器的两次交互
- 2. HTML 表单
  - 2.1 表单的关键属性
    - 2.1.1 表单源码
    - 2.1.2 浏览器渲染
  - 2.2 GET 提交
  - 2.3 POST 提交
  - 2.4 GET 与 POST 的语义
  - 2.5 URL 编码
  - 2.6 GET 参数与路由
  - 2.7 常见正文格式
- 3. URI 从文件切换到服务
  - 3.1 路由需要方法和路径
    - 3.1.1 POST 路径路由
    - 3.1.2 URI 功能路由
    - 3.1.3 请求数据
  - 3.2 服务注册
  - 3.3 登录处理
- 4. RESTful 动态接口
  - 4.1 资源与方法
  - 4.2 JSON 请求
  - 4.3 动态服务也是 HTTP 客户端

1. 从静态文件到动态服务

静态网站主要返回已有文件，浏览器通常通过 GET 请求资源。动态网站需要执行服务器逻辑，例如登录、注册、搜索、支付和下单，因此客户端必须把用户输入一起提交给服务器。

```
静态请求：GET /image/logo.png
动态请求：POST /login
```

动态请求仍然使用 HTTP，只是 URI 不再一定对应文件，正文也可能携带业务参数。

前置阅读：

- Linux网络--Http请求与响应
- Linux网络--Http静态资源服务器

1.1 浏览器的两次交互

一次登录通常至少包含两次请求：

```
GET /login  -> 服务器返回登录页面
POST /login -> 浏览器提交用户名和密码
```

GET 取得页面，POST 提交数据。服务器处理完成后，可能直接返回结果，也可能通过 303 重定向到用户主页。

2. HTML 表单

2.1 表单的关键属性

```
<form action="/login" method="post">
  <label for="username">用户名：</label>
  <input type="text" id="username" name="username">

  <label for="password">密码：</label>
  <input type="password" id="password" name="password">

  <input type="submit" value="Submit">
</form>
```

- `action`：请求目标 URI。
- `method`：GET 或 POST 等提交方法。
- `name`：请求参数的键。
- 用户输入：请求参数的值。

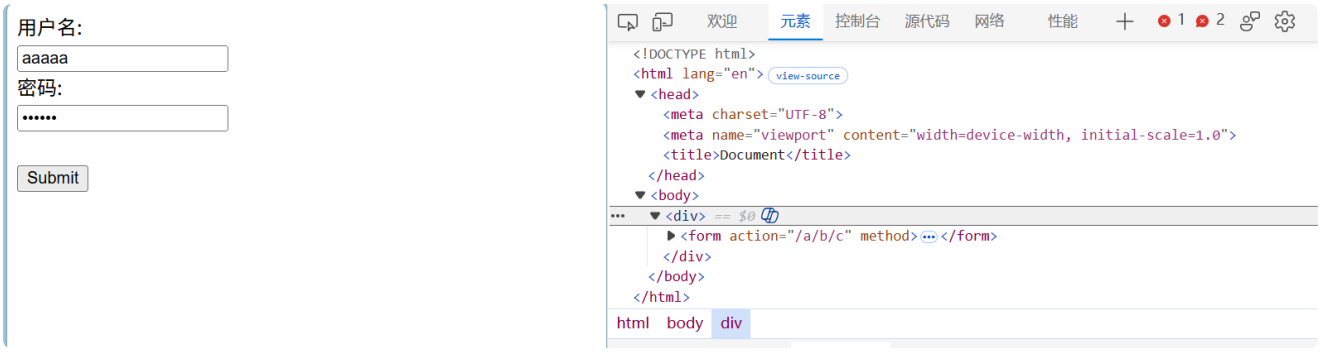
浏览器负责读取表单控件、编码字段、构造请求并向服务器发送。缺少 `name` 的控件通常不会进入表单数据。

2.1.1 表单源码

```html
<body>
    <div>
        <form action="/a/b/c" method="get"">
            <label for="name">用户名:</label><br>
            <input type="text" id="fname" name="name"><br>
            <label for="passwd">密码:</label><br>
            <input type="password" id="lname" name="passwd"><br><br>
            <input type="submit" value="Submit">
        </form>
    </div>
</body>
```

图：表单的 `action`、`method` 和两个输入字段。

2.1.2 浏览器渲染



图：浏览器将同一份 HTML 渲染成输入框和提交按钮，右侧展示了对应 DOM。

2.2 GET 提交

GET 表单把字段编码到查询字符串：

```
表单字段：name=aaaaa，passwd=aaaaaa
请求目标：/a/b/c?name=aaaaa&passwd=aaaaaa
```

浏览器构造的请求行近似为：

```
GET /a/b/c?name=aaaaa&passwd=aaaaaa HTTP/1.1
Host: 192.0.2.10:8080
```

```
 GET /a/b/c?name=aaaaa&passwd=111111 HTTP/1.1
Host: 82.156.75.119:8080
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://82.156.75.119:8080/login.html
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
```

图：浏览器把表单字段拼接到 action 后面，形成带查询参数的 GET 请求。

```
[2026-09-14 13:56:23.488744][DEBUG][3298775][TcpServer.hpp][36]-get a new link,socket address: 111.42.43.67 64576 sockfd: 18
[2026-09-14 13:56:23.489097][DEBUG][3299966][TcpServer.hpp][68]-inbuffer:
 GET /a/b/c?name=222222&passwd=2222222 HTTP/1.1
Host: 82.156.75.119:8080
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://82.156.75.119:8080/login.html
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
```

图：服务器接收到的 GET 请求，参数位于请求行中的查询字符串。

GET 参数可能出现在地址栏、浏览器历史、服务器日志和 Referer 中，因此适合查询和过滤，不适合密码、Token 等高敏感值。

2.3 POST 提交

POST 把参数放在正文：

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30

username=alice&password=123456
```

```
[2026-09-14 13:52:12.822869][DEBUG][3298775][TcpServer.hpp][36]-get a new link,socket address: 111.42.43.67 51770 sockfd: 11
[2026-09-14 13:52:12.824190][DEBUG][3298896][TcpServer.hpp][68]-inbuffer:
 POST /a/b/c HTTP/1.1
Host: 82.156.75.119:8080
Connection: keep-alive
Content-Length: 24
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
Origin: http://82.156.75.119:8080
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://82.156.75.119:8080/login.html
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

name=44444&passwd=aaaaaa
```

图：服务器接收到的 POST 请求，参数位于空行之后的正文。

POST 参数不出现在地址栏，但普通 HTTP 仍是明文。只有 HTTPS 才能保护传输中的机密性和完整性。

2.4 GET 与 POST 的语义

维度	GET	POST
参数位置	查询字符串	请求正文
地址栏可见	通常可见	通常不可见
历史与日志	更容易留下参数	服务器日志仍可能记录正文
幂等性	应保持幂等	通常不保证
典型用途	查询、获取资源	创建、提交、状态变化

HTTP 没有规定 URL 的绝对最大长度，实际限制来自浏览器、代理和服务器。选择 POST 的主要依据是操作语义和正文设计。

2.5 URL 编码

参数中的空格、中文、`&`、`=`、`?`、`#`、`/` 等字符需要编码：

```
C++ 教程
C%2B%2B%20%E6%95%99%E7%A8%8B
```

非 ASCII 字符先按 UTF-8 编码成字节，再逐字节转成 `%HH`。在 `application/x-www-form-urlencoded` 中，空格也可能写成 `+`。

解析顺序应是：

```
先按 & 区分字段
  -> 再按 = 区分键和值
  -> 分别进行 URL 解码
```

如果先整体解码，参数值中原本编码过的 `&` 或 `=` 会被误认为新的分隔符。

2.6 GET 参数与路由

GET 表单提交后，服务器可能看到：

```
/Login?name=hhhhhh&passwd=111111
```

但注册的路由可能只是：

```
/Login
```

因此必须先分离路径和查询字符串，再匹配路由：

```
const auto question = target.find('?');
const std::string path =
    question == std::string::npos ? target : target.substr(0, question);
const std::string query =
    question == std::string::npos ? "" : target.substr(question + 1);
```

```
[2026-09-14 16:42:23.714966][DEBUG][3343563][TcpServer.hpp][36]-get a new link,socket address: 111.55.112.116 28239 sockfd: 7
[2026-09-14 16:42:23.724584][DEBUG][3343856][TcpServer.hpp][68]-inbuffer:
 GET /Login?name=hhhhhh&passwd=666666666 HTTP/1.1
Host: 82.156.75.119:8080
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://82.156.75.119:8080/login.html
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
```

图：GET 表单提交后，`/Login` 与查询参数连在同一个请求目标中。

2.7 常见正文格式

Content-Type	用途	典型内容
`application/x-www-form-urlencoded`	普通 HTML 表单	`a=1&b=hello`
`multipart/form-data`	文件上传	多个 part 和文件字节
`application/json`	API 请求	`{"username":"alice"}`
`text/plain`	纯文本	无结构文本
`application/octet-stream`	原始二进制	文件字节流

服务器应根据 `Content-Type` 选择解析器，不能把任何正文都按表单键值解析。

3. URI 从文件切换到服务

后端可以从两个角度理解请求目标：

```
/image/logo.png -> Web 根目录中的文件
/login          -> 动态登录服务
/api/users/42   -> RESTful 业务资源
```

判断流程可以是：

```
解析 method 和 target
  -> 分离 path 和 query
  -> 先检查动态路由表
  -> 路由命中：执行处理函数
  -> 未命中：按静态文件路径处理
```

对于动态服务，URI 只是路由键；对于静态资源，URI 才映射到文件路径。

3.1 路由需要方法和路径

只使用 URI 作为键不够，因为同一个 `/login` 可以支持：

```
GET  /login -> 返回登录页面
POST /login -> 处理登录表单
```

更合适的路由键是：

```
(method, normalized_path)
```

例如：

```
("GET",  "/login") -> showLoginPage
("POST", "/login") -> login
("GET",  "/api/users") -> listUsers
```

3.1.1 POST 路径路由

```
[2026-09-14 14:26:23.001607][DEBUG][3308075][TcpServer.hpp][36]-get a new link,socket address: 111.42.43.67 56633 sockfd: 8
[2026-09-14 14:26:23.002007][DEBUG][3308289][TcpServer.hpp][68]-inbuffer:
 POST /Login HTTP/1.1
Host: 82.156.75.119:8080
Connection: keep-alive
Content-Length: 28
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
Origin: http://82.156.75.119:8080
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://82.156.75.119:8080/login.html
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
```

图：POST 请求 `/Login` 被路由到对应的登录服务。

3.1.2 URI 功能路由

```
[2026-09-14 14:37:06.243047][DEBUG][3310986][TcpServer.hpp][36]-get a new link,socket address: 111.42.43.67 59553 sockfd: 6
[2026-09-14 14:37:06.243449][DEBUG][3311118][TcpServer.hpp][68]-inbuffer:
 POST /Login HTTP/1.1
Host: 82.156.75.119:8080
Connection: keep-alive
Content-Length: 25
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
Origin: http://82.156.75.119:8080
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://82.156.75.119:8080/login.html
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

name=hhhhhh&passwd=111111
```

图：服务器根据 URI 找到已注册的动态处理函数。

3.1.3 请求数据

```
_versionHTTP/1.1
_uriwwwzhangling/Login
_methodPOST
->Referer # http://82.156.75.119:8080/login.html
->Origin # http://82.156.75.119:8080
->User-Agent # Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
->Accept-Language # zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
->Accept-Encoding # gzip, deflate
->Upgrade-Insecure-Requests # 1
->Cache-Control # max-age=0
->Accept # text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
->Content-Length # 24
->Connection # keep-alive
->Content-Type # application/x-www-form-urlencoded
->Host # 82.156.75.119:8080


name=hhhhh&passwd=aaaaaa
data is: name=hhhhh&passwd=aaaaaa
```

图：请求反序列化后，处理函数可以读取方法、URI、Header 和正文参数。

3.2 服务注册

可以用映射保存路由：

```
using Handler = std::function<HttpResponse(const HttpRequest&)>;

std::unordered_map<std::string, Handler> routes;
routes["GET /"] = handleIndex;
```

```
routes["GET /login"] = showLogin;
routes["POST /login"] = login;
routes["GET /api/users"] = listUsers;
```

注册时需要统一路径格式：

- 去除查询字符串。
- 规范化连续斜杠。
- 区分大小写策略。
- 处理末尾斜杠策略。
- 避免把未规范化路径直接作为键。

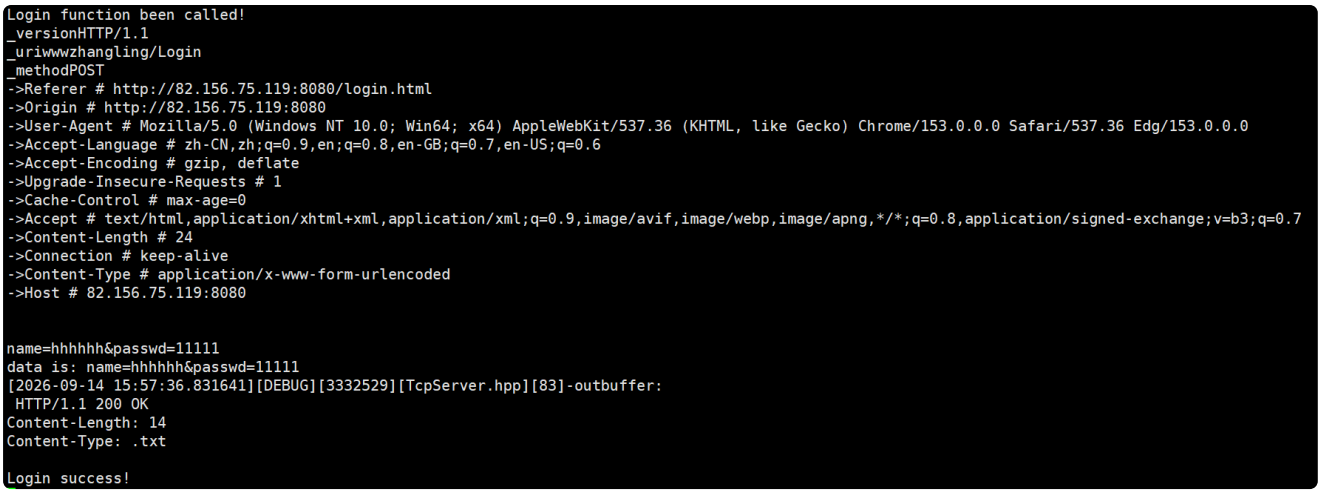
3.3 登录处理

```
POST /login
  -> 读取正文
  -> 按 Content-Type 解析参数
  -> 校验用户名和密码
  -> 创建 Session 或 Token
  -> 设置响应状态、Header 和正文
  -> 序列化并发送
```

登录函数可以返回纯文本：

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 14

Login success!
```



图：登录处理函数被调用，并返回构造好的 HTTP 响应。

真实系统应使用密码哈希、登录限流和 HTTPS，不能直接比较或返回明文密码。

4. RESTful 动态接口

4.1 资源与方法

请求	语义
`GET /api/users`	查询用户列表
`POST /api/users`	创建用户
`GET /api/users/42`	查询指定用户
`PUT /api/users/42`	替换用户
`PATCH /api/users/42`	部分更新用户
`DELETE /api/users/42`	删除用户

接口名称描述资源，HTTP 方法描述操作。状态码表达结果，例如创建成功可以使用 201，成功但无正文可以使用 204。

4.2 JSON 请求

```
POST /api/users HTTP/1.1
Content-Type: application/json
Content-Length: 41

{"username":"alice","age":24}
```

服务器需要：

1. 确认 `Content-Type`。
2. 读取完整字节数。
3. 按 UTF-8 解码。
4. 解析 JSON。
5. 校验字段类型、范围和权限。
6. 返回 JSON 响应和正确状态码。

错误请求应返回合适的 4xx，而不是把解析失败包装成 200。

4.3 动态服务也是 HTTP 客户端

登录、搜索或订单服务可能需要继续访问数据库、Redis、另一个 HTTP API 或大模型服务：

```
浏览器 -> 当前 HTTP 服务 -> 下游服务
       <- 当前 HTTP 服务 <-
```

当前服务器在上游是服务端，在下游又可以作为客户端。转发请求时必须保留必要的 Header、超时、错误码和响应类型。

下一篇分析浏览器为什么会产生大量请求，以及 HTTP/1.0、1.1、HTTP/2 和 HTTP/3 的连接差异：Linux网络--Http浏览器与连接演进